

ProjectHub

Plan działania

Etap 1: Analiza i projektowanie architektury

Zanim wygenerujemy pierwszą linię kodu, musimy ustalić ramy teoretyczne. Ten etap uchroni nas przed błędami architektonicznymi w późniejszym czasie.

1. **Model danych (ERD):** Zaprojektujemy strukturę relacyjnej bazy danych dla minimum 5 powiązanych encji tworzących spójny model biznesowy. Standardowo będą to: User, Workspace, Project, Task, Comment.
2. **Role i uprawnienia:** Zdefiniujemy minimum 2 role użytkowników (np. USER i ADMIN). Ustalimy politykę dostępu (Authorization) ograniczającą dostęp do wybranych endpointów.
3. **Reguły biznesowe:** Zdefiniujemy i dopracujemy minimum 3 odrębne reguły biznesowe wykraczające poza proste operacje CRUD, np. zakaz zamykania zadań bez wykonawcy lub weryfikacja uprawnień przy przenoszeniu zadań.
4. **Wybór architektury:** Przyjmiemy podział aplikacji na warstwy (np. obsługa requestów, logika biznesowa, dostęp do danych). W .NET zastosujemy architekturę wielowarstwową (N-Tier) lub Vertical Slice Architecture.

Etap 2: Inicjalizacja i konfiguracja środowiska

Przygotowanie repozytorium oraz niezbędnej infrastruktury narzędziowej.

1. **Repozytorium:** Utworzenie struktury katalogów na GitHub, dodanie globalnego pliku .gitignore dla .NET, Node.js oraz JetBrains Rider.
2. **Konteneryzacja bazy danych:** Stworzenie pliku docker-compose.yml uruchamiającego kontener z bazą PostgreSQL. Umożliwi to niezależne i powtarzalne środowisko deweloperskie.
3. **Backend:** Wygenerowanie pustego projektu ASP.NET Core Web API w JetBrains Rider. Skonfigurowanie wstrzykiwania zależności (DI) oraz przygotowanie plików konfiguracyjnych (np. appsettings.json), pamiętając o braku bezpośrednio zapisanych sekretów.
4. **Frontend:** Inicjalizacja projektu React z TypeScript (np. za pomocą Vite).

Etap 3: Warstwa dostępu do danych i autoryzacja (Backend)

Podłączenie aplikacji do bazy danych oraz zabezpieczenie dostępu.

1. **Entity Framework Core:** Implementacja modeli na podstawie zaprojektowanego ERD. Konfiguracja kontekstu bazy danych (DbContext) z wykorzystaniem Fluent API.
2. **Migracje:** Zastosowanie mechanizmu migracji schematu bazy danych i wygenerowanie początkowego stanu.
3. **Identity i JWT:** Implementacja uwierzytelniania użytkownika (Authentication) w oparciu o tokeny JWT. Skonfigurowanie globalnych autoryzacji w potoku ASP.NET Core.

Etap 4: Logika biznesowa i REST API (Backend)

Realizacja głównego rdzenia aplikacji. Kod HTTP będzie w pełni oddzielony od logiki domenowej.

1. **Wzorzec DTO:** Zastosowanie Data Transfer Objects do oddzielenia modelu API od modelu bazy danych.
2. **Endpointy i Walidacja:** Implementacja endpointów (metody HTTP, statusy) m.in. dla listowania, dodawania, edycji i usuwania danych. Dodanie mechanizmu paginacji, wyszukiwania i filtrowania dla zasobów. Zastosowanie biblioteki takiej jak FluentValidation do walidacji danych wejściowych.
3. **Globalna obsługa błędów:** Wdrożenie Middleware'u przechwytyjącego wyjątki i tłumaczącego je na ustandaryzowane odpowiedzi HTTP z odpowiednimi kodami.
4. **Dokumentacja API:** Skonfigurowanie narzędzia Swagger / OpenAPI, co znacząco ułatwi późniejsze pisanie zapytań we wbudowanym w Ridera pliku .http.

Etap 5: Implementacja wymagań zaawansowanych

Zintegrowanie mechanizmów pozwalających na uzyskanie najwyższej oceny.

1. **Mechanizm asynchroniczny:** Wykorzystanie systemu zdarzeń (np. MediatR lub asynchronicznych BackgroundServices) do obsługi np. powiadomień lub generowania statystyk zadań.
2. **Element rozszerzony:** Implementacja WebSocketów (SignalR). Pozwoli to na bezodświeżeniową, natychmiastową aktualizację stanu tablicy Kanban u wszystkich użytkowników pracujących w danym obszarze roboczym.

Etap 6: Aplikacja kliencka (Frontend)

Połączenie gotowego REST API z interfejsem graficznym.

1. **Integracja z API:** Skonfigurowanie klienta HTTP (np. Axios) do komunikacji z REST API.
2. **Autoryzacja po stronie klienta:** Obsługa logowania, przechowywania tokena JWT oraz zabezpieczenie routingu w React.
3. **Implementacja widoków:** Opracowanie głównych interfejsów m.in. logowania, widoku projektów oraz samej tablicy Kanban (możliwość listowania, dodawania, edycji, usuwania danych oraz obsługa błędów API z poziomu interfejsu).

Etap 7: Zapewnienie jakości (Testowanie)

Spełnienie rygorystycznych wymogów testowych. Zostaną napisane za pomocą xUnit.

1. **Testy jednostkowe (Unit Tests):** Napisanie minimum 5 testów skupiających się wyłącznie na wyizolowanej logice biznesowej (weryfikacja, czy zaimplementowane reguły działają niezależnie od bazy danych i HTTP).
2. **Testy integracyjne:** Zbudowanie minimum 3 testów integracyjnych weryfikujących działanie wybranych ścieżek REST API (od przyjęcia zapytania do komunikacji z in-memory DB lub Testcontainers).

Etap 8: Wdrożenie i dokumentacja (Finał)

Dopięcie formalności niezbędnych do oddania projektu.

1. **Konteneryzacja całości:** Aktualizacja pliku docker-compose.yml, aby podnosił osobne kontenery zarówno dla aplikacji backendowej, jak i bazy danych.
2. **Plik README.md:** Stworzenie wyczerpującej instrukcji opisującej projekt, technologie, instrukcję uruchomienia, diagram ERD i zwięzłą dokumentację.
3. **Demo Video:** Nagranie materiału wideo (3-5 min) demonstrującego działanie systemu i wyjaśniającego przepływ danych, model i reguły biznesowe.